



Sintaxis y Semántica del Lenguaje

Trabajo Práctico Integrador: Diseño e implementación de Lexer y Parser

Intérprete JSON y Traductor a HTML

Ciclo lectivo: 2025

Equipo docente:

Director de Cátedra

Nombre y Apellido: Ing. Gabriela P. TOMASELLI

Categoría docente: Profesora Asociada

e-mail: gabriela.tomaselli@gmail.com

Docentes

Ing. Gabriela P. TOMASELLI

e-mail: gabriela.tomaselli@gmail.com

Ing. Rodrigo VIGIL

e-mail: rodrigovigil@gmail.com

Ing. Nicolas G. TORTOSA

e-mail: nicotortosa@gmail.com

Ing. Juliana I. TORRE

e-mail: julitorre025@gmail.com



Índice de Contenido

1. Introducción	3
Competencias	3
Objetivo	3
2. Elaboración y entrega	3
Presentaciones.	3
Metodología de entregas parciales y final.	4
3. Conformación de equipos	4
Matriz de habilidades	4
Identidad	4
Alianza de equipo	5
4. Características de lenguaje JSON	5
Introducción	5
Tipos de Datos:	5
◦ URL	6
E-mail	6
Atributos con valores específicos	6
	7
Estructura del documento	7
Listas y objetos del documento	7
Consideraciones:	7
5. Traducción a HTML	9
6. Control de errores	9
6.1. Modos de ejecución del intérprete	9
• Modo interactivo - Analiza mientras escribe	9
• Ejecución desde un archivo	9
7. Documentación del trabajo	10
Informe técnico	10
Video Educativo / Promocional:	11
8. Presentación y Criterios de evaluación	11
Documentación: 40 %	11
Funcionamiento del intérprete (software): 60 %	11
9. Ejemplos	11
Básico	11



1. Introducción 1

Competencias

El presente trabajo práctico integrador pretende desarrollar las siguientes competencias de la asignatura:

Capacidad para conocer los fundamentos teóricos de los lenguajes de programación y las técnicas de procesamiento léxico, sintáctico asociadas, y saber aplicarlas para la creación, diseño y procesamiento de lenguajes de programación.

Capacidad de diseño, y habilidad instrumental necesaria para llevar a cabo proyectos de cierta envergadura, cuya complejidad exige la utilización de conocimientos adquiridos en diversas asignaturas a lo largo de la carrera.

Aptitud y actitud para trabajar en equipo.

Objetivo

Utilizando algún lenguaje de programación o algún generador de lexer y parser para determinado lenguaje de programación deberán construir los analizadores léxico y sintáctico que permita analizar, validar y transformar un documento JSON que describe y almacena información de equipos y proyectos.

El parser a construir recibe un archivo en formato JSON y deberá indicar si está bien construido (adecuado al formato indicado, sin errores) de otra manera indicar los errores; adicionalmente a medida que analiza el documento deberá transformar el contenido en un documento HTML.

El analizador lexicográfico es un módulo que recibe una secuencia de caracteres que componen el documento a analizar y lo convierte lógicamente en una secuencia de tokens. El analizador sintáctico recibe la secuencia de tokens que le entrega el analizador lexicográfico y verifica si es correcta de acuerdo a una determinada gramática. Adicionalmente puede generar diversas salidas de acuerdo a criterios especificados.

Hay dos grupos de herramientas que se pueden usar para generar los analizadores:

1) Se utilizan expresiones regulares y autómatas finitos para el análisis léxico que la secuencia pueda ser generada por la gramática del lenguaje.

El analizador lexicográfico y la técnica LALR para el análisis sintáctico. Ejemplos de esto son lex y yacc, que generan código C o C++, o JLex y CUP, que generan código Java. flex y bison son implementaciones libres y gratuitas de lex y yacc.

2) El otro utiliza la técnica LL(k) tanto para el análisis léxico como para el sintáctico, generando parsers descendentes recursivos. Ejemplos son JavaCC, que genera código Java, y ANTLR, que está escrito en Java pero puede generar código Java, C++ python o C#. ANTLR se puede conseguir en <http://www.antlr.org/>.

https://en.wikipedia.org/wiki/Comparison_of_parser_generators

2. Elaboración y entrega

Presentaciones.

Se prevén cuatro instancias de presentación:

. **entrega inicial:** Conformación de equipo (matriz, identidad, alianza).

Domingo 13 de Abril de 2025

- **1er entrega:** Primera documentación del proyecto; gramática a generar.

Domingo 04 de Mayo de 2025

. **2da entrega:** Presentación del lexer o scanner que reconozca los tokens del lenguaje.

Domingo 8 de Junio de 2025



. **3ra entrega final:** Presentación de trabajo completo. Lexer y parser, incluye la presentación del trabajo ante la clase con una exposición de no más de 20 minutos.-

Domingo 06 de Julio de 2025

Metodología de entregas parciales y final.

o Un archivo comprimido deberá ser "subido" a la tarea correspondiente dentro del curso en el campus virtual.

o Dicho archivo comprimido deberá contener, según corresponda:

directorio **doc**. Documentación del trabajo (véase el apartado n° 6)

directorio **src**. archivos fuente del proyecto (flex, bison, python, archivos de C (".c", ".h")

directorio **bin**. Archivos binarios listos para ser ejecutados. (lexer y parser)

directorio **prueba**. Archivos de ejemplo de pseudocódigo con la extensión ".json"

3. Conformación de equipos

Todos los alumnos deberán completar la [encuesta inicial 2025](#), a fin de conocer habilidades y capacidades de cada alumno, la misma estará disponible hasta el día *28 de Marzo de 2025*, luego se publicarán los resultados, a fin de que cada equipo pueda conformarse de la mejor manera.

El trabajo se realizará de forma grupal en equipos de hasta cinco (5) alumnos, donde cada integrante deberá realizar y presentar una parte del trabajo durante la presentación final.

Cada equipo debe ser responsable, autónomo y adaptable.

Cada equipo tendrá la libertad de definir cómo trabajan, pero debe haber un líder que de las pautas de qué se debe hacer, y en qué condiciones se espera que se haga o cuales son los resultados esperados, estará representado por un delegado, encargado de enviar las entregas parciales y el trabajo final al campus virtual, enviará consultas o dudas, etc.

Cada equipo tendrá un docente designado como tutor encargado del seguimiento del trabajo, responderá inquietudes.

Matriz de habilidades

Cada equipo deberá conformar su propia **matriz de habilidades**:

Deberán armar una tabla y escribir en las columnas las principales competencias necesarias para completar el trabajo. En las filas de esa misma tabla los nombres de las personas del equipo.

Luego:

Puntúa dentro de la tabla el grado de habilidad de cada persona respecto a cada competencia:

→ Poner un asterisco * o estrella ★ en donde la persona es experta.

→ Poner un punto ○ donde la persona no es experta pero podría desenvolverse con dicha competencia (le interesa, es bueno en eso aunque no es su mayor interés, o está dispuesto a aprenderlo).

Dejar en blanco competencias que una persona desconozca por completo (o se niegue a participar en ellas).

Ejemplo de Competencia: Inglés, buscar info, programar, idear soluciones, escribir documentación, redes sociales, edición video, cebar mates, etc.-

Identidad

Cada equipo deberá proponer un nombre y logo para identificarse . Algo que represente al equipo y lo haga diferente a los demás.

Un equipo sin identidad nunca será un gran equipo y puede, incluso, que no sea ni equipo.



Alianza de equipo

Un breve enunciado donde el equipo define como trabajará y se organizará, a que se comprometen, cuales son sus metas y valores, los objetivos comunes.

Por ejemplo, si el trabajo requerirá de juntarse una vez a la semana fuera de la universidad, establecer en qué días y horarios podría ser, o cómo será la organización de esa reunión. Y si necesitaremos comunicarnos fuera de clase, cuáles serán esos canales.

4. Características de lenguaje JSON

Introducción

JSON, abreviatura de "JavaScript Object Notation" (Notación de Objetos de JavaScript), es un formato de intercambio de datos ligero y fácil de leer. Surgió como una alternativa más simple y legible que XML (Extensible Markup Language) para la transmisión de datos entre aplicaciones web. Aunque su nombre hace referencia a JavaScript, JSON es un formato independiente de cualquier lenguaje de programación y es ampliamente utilizado en diversas tecnologías y plataformas.

La estructura de datos en JSON se basa en dos tipos principales: objetos y matrices. Un objeto JSON es una colección no ordenada de pares clave-valor, donde las claves son cadenas de texto y los valores pueden ser cadenas de texto, números, booleanos, otros objetos JSON, matrices o valores nulos. Por otro lado, una matriz JSON es una secuencia ordenada de valores, que pueden ser de cualquier tipo permitido en JSON.

La sintaxis de JSON es muy similar a la notación de objetos de JavaScript, lo que facilita su comprensión y manipulación para desarrolladores familiarizados con ese lenguaje. Consiste en pares de clave-valor encerrados entre llaves {} para objetos y corchetes [] para listas, separados por comas y con los valores representados según su tipo específico.

JSON es utilizado en una variedad de contextos, incluyendo la comunicación de datos entre el cliente y el servidor en aplicaciones web, el almacenamiento de configuraciones y datos estructurados, y como formato de intercambio en servicios de API (Interfaz de Programación de Aplicaciones), debido a su simplicidad, eficiencia y facilidad de lectura y escritura tanto para humanos como para máquinas.

Tipos de Datos:

- String
 - Compuesta por letras, números, signos de puntuación y caracteres especiales.
 - Se deben encerrar entre comillas dobles.
- Integer
 - Números enteros positivos.
 - No llevan comillas.
- Float
 - Número real positivo.
 - A los efectos de este práctico permitir solo 2 posiciones decimales.
 - No llevan comillas.
 - La parte decimal se separa con el punto.
- Bool
 - Booleano, valores posibles **true** o **false**
 - No llevan comillas.
- Date
 - Formato de fecha de tipo "YYYY-MM-DD" (año-mes-día)
 - A los efectos de este práctico limitaremos las fechas de la siguiente manera
 - Año debe ser un número entre 1900 y 2099.



- Mes debe ser un número entre 1 y 12
- Día debe ser un número entre 1 y 31
- No es necesario validar correspondencia entre día y mes (ej. 31 de feb).
- Se deben encerrar entre comillas dobles.
- **URL**

Nota: Una URL (**Uniform Resource Location**) es la dirección concreta de un recurso en Internet.
La sintaxis completa de una URL es: **protocolo://dominio:puerto/ruta**

- El **protocolo o esquema de red**. Hace referencia al nombre de protocolo de red necesario para poder alcanzar el recurso al que hace referencia la URL. Protocolos habituales son:
 - **http://** (para recursos de la web)
 - **https://** (para recursos de la web contenidos en un servidor seguro)
- **dominio**. Nombre completo en Internet de la máquina (o la red) que posee el recurso en forma de nombre de dominio. No distinguen entre mayúsculas y minúsculas.
- **Puerto. Opcional**. Puerto por el que se debe conectar con el servidor para obtener el recurso. Si no se indica (que es lo habitual) se toma el puerto por defecto. Por ejemplo en *http* el puerto por defecto es el *80*. Si queremos usar uno en particular se indica tras el servidor poniendo dos puntos y el puerto.
- **Ruta. Opcional**. Indica el recorrido dentro de la máquina remota que hay que hacer a través de los directorios para llegar al recurso que queremos. Se pone después del servidor. Ejemplos:
 - */index.html* Accede a la página *index.html* situada en el directorio raíz.
 - */imagenes/paisajes/foto001.jpg* Accede a la imagen *foto001.jpg* dentro del directorio *paisajes* dentro, a su vez, del directorio *imagenes*.
- Los únicos caracteres permitidos en URL son letras, números, guión medio, guión bajo y punto. además de los caracteres reservados: **#, /, :**
- Se deben encerrar entre comillas dobles.

E-mail

un correo electrónico es una secuencia de caracteres que describe el patrón de un correo electrónico válido. El formato de un correo electrónico válido es: **Nombre de usuario, Símbolo arroba (@), Dominio, Extensión**. En esta expresión, los caracteres que se pueden usar para el *nombre y dominio* son: Letras ,Dígitos ,Guiones bajos ,Puntos ,Signos de más (+), Signos de menos (-)
El símbolo **@** es obligatorio en todas las direcciones de correo electrónico válidas.
la *Extensión* admite letras de longitud 2 hasta 4.

- Se deben encerrar entre comillas dobles.

Atributos con valores específicos

Si bien JSON solo valida en base al tipo de dato del atributo, a los efectos de este práctico algunos atributos tendrán también restricciones de formato.

cargo (integrante) = Tipo de dato string, una de las siguientes opciones:

- Product Analyst | Project Manager | UX designer | Marketing | Developer | Devops | DB admin

estado (proyecto) = Tipo de dato string, una de las siguientes opciones:

- To do | In progress | Canceled | Done | On hold



Estructura del documento

Listas y objetos del documento

- **Opcionales:** Un nombre de elemento sin adornos indica que un elemento debe ocurrir exactamente **una vez**, Cuando un elemento es opcional se representa con **?**
- **Lista** implica que puede haber más de un elemento de ese objeto.
- *Importante, todas las keys son sensibles a mayúsculas y deben respetarse como tal.*

Objeto JSON con los atributos: equipos, version?, firma_digital?

equipos = Lista de objetos equipo.

equipo = Objeto con los atributos: nombre_equipo, identidad_equipo, dirección?, link?, carrera, asignatura, universidad_regional, alianza_equipo, integrantes, proyectos.

dirección = Objeto con los atributos: calle, ciudad, país.

integrantes = Lista de objetos integrante.

integrante = Objeto con los atributos: nombre, edad?, cargo, foto, email, habilidades, salario, activo.

proyectos = Lista de objetos proyecto.

proyecto = Objeto con los atributos: nombre, estado, resumen, tareas, fecha_inicio, fecha_fin, video, conclusion.

tareas = Lista de objetos tarea.

tarea = Objeto con los atributos: nombre, estado, resumen, fecha_inicio?, fecha_fin?

Consideraciones:

- El JSON completo debe estar encerrado entre llaves. {}
- Todos los **objetos** se encierran entre llaves. {}
- Las key de un objeto se encierran entre comillas, mientras que los valores llevan o no comillas dependiendo del tipo de datos (Ver apartado Tipo de datos)
- Las listas se encierran entre corchetes []
- Por simplicidad no se contempla dentro del documento vocales acentuadas ni eñe (ñ).
- Luego de cada elemento siempre hay una coma **excepto** antes de un cierre de objeto o lista.
- Cuando un elemento es OPCIONAL puede no estar, puede tener valor **null** o puede ser el objeto vacío ({}, [])
- Los atributos de los objetos *equipo*, *integrante* y *proyecto* respetaran el orden dado, mientras que para todos los demás objetos los atributos pueden aparecer en cualquier orden

```
{ "equipos": [  
  {  
    "nombre_equipo": string,  
    "identidad_equipo": url,  
    "link": url,  
    "asignatura": string,  
    "carrera": string,  
    "universidad_regional": string,  
    "dirección": {  
      "calle": string,
```



```
"ciudad": string,  
"país": string  
},  
"alianza_equipo": string,  
"integrantes": [  
  {  
    "nombre": string,  
    "edad": integer,  
    "cargo": string,  
    "foto": url,  
    "email": email,  
    "habilidades": string,  
    "salario": float,  
    "activo": bool  
  } ],  
"proyectos": [  
  {  
    "nombre": string,  
    "estado": string,  
    "resumen": string,  
    "tareas": [  
      {  
        "nombre": string,  
        "estado": string,  
        "resumen": string,  
        "fecha_inicio": date,  
        "fecha_fin": date  
      } ],  
    "fecha_inicio": date,  
    "fecha_fin": date,  
    "video": url,  
    "conclusion": string,  
  } ]  
}  
],  
"version": string,  
"firma_digital": string  
}
```



5. Traducción a HTML

Traducir el documento, generando un archivo de texto HTML ,transformando algunos elementos en etiquetas HTML. Formateado de la siguiente manera:

- Nombre de archivo: igual al archivo fuente JSON pero con extensión HTM o HTML.
- Equipo: cada equipo debe contenerse en un div **<div>** con un borde de 1px gris y un padding de 20px;.
- Nombre de la equipos : encerrados entre tags: **<h1>**
- Datos de los equipos : deben traducirse como parrafo **<p>**
- Todos items **Link** deberá traducirse como tag html link **a**, redireccionando al link correspondiente.
- Nombre de los Integrantes : encerrados entre tags: **<h2>**
- Datos de los Integrantes : deben traducirse como listas html **ul** y los nombres como item **li**
- Nombre de los proyectos : encerrados entre tags: **<h3>**
- Datos de los proyectos : deben traducirse como listas html **ul** y los nombres como item **li**
- Tareas: deben traducirse como tabla debajo del proyecto correspondiente, donde cada atributo es una columna.

6. Control de errores

El intérprete deberá detectar, capturar y controlar errores indicando tipo de error , número de línea y cadena que generó el error.

- Léxicos:
 - Etiquetas faltantes o mal escritas.
 - Utilización de símbolos no permitidos.
 - Etc.
- Sintácticos:
 - Etiquetas que aparecen en lugares no debidos.
 - Etiquetas o elementos obligatorios faltantes.
 - Elementos con atributos incompatibles.
 - Etc.
 - Observación : Se valorará la utilización de “reglas de producción de control de errores” que no generen conflictos.
- De ejecución
 - archivo de entrada inexistente o con una extensión incorrecta.
 - Etc.

6.1. Modos de ejecución del intérprete

El intérprete deberá ejecutarse de dos formas diferentes:

• **Modo interactivo - Analiza mientras escribe**

- Se escribe e interpreta manualmente desde un terminal de texto.
- Se utilizará el carácter de fin de archivo para terminar la ejecución: *Control + D*
- Útil para reconocer tokens, depuración y pruebas.
- **Obligatorio para tercera presentación (lexer)**

• **Ejecución desde un archivo**

- Se interpretarán las sentencias de un archivo pasado como argumento desde la línea de comandos , o abierto por opciones gráficas.
- El archivo deberá tener la extensión “.json”
- **Obligatorio para presentación final (parser)**



7. Documentación del trabajo

Se deberá documentar todo el desarrollo de trabajo; recolectando, redactando, describiendo y exponiendo todo lo realizado.

Para ello deberán realizar y presentar:

Informe técnico

Documento principal del trabajo, que resume lo realizado desde el inicio, desde la conformación del grupo hasta la entrega final y exposición. Escrito con sus palabras (No solo wikipedia, ChatGPT) , resumiendo y promocionando el trabajo desarrollado, con las siguientes características:

- Portada
 - o Título del proyecto desarrollado
 - o Nombre y apellidos de las personas que forman el grupo
 - o Nombre de la asignatura
 - o Nombre de la carrera
 - o Primer cuatrimestre
 - o Curso académico: 2025
 - o Universidad y regional
 - o Lugar y fecha
- Índice
 - o Las páginas deberán estar numeradas.
- Introducción
 - o Breve descripción del trabajo realizado y de las partes del documento.
 - o Descripción de cómo se implementó la solución.
 - o Información y requerimientos de software para ejecutar y compilar el tp (versiones de compiladores, herramientas, plataforma, etc).
- Conformación del Grupo: Integrantes, matriz, identidad, alianza.
- Gramática.
 - o Descripción de cada regla de la gramática libre de contexto a utilizar.
 - # Símbolos de la gramática
 - # Símbolos terminales (componentes léxicos)
 - # Símbolos no terminales
 - #Reglas de producción de la gramática
 - # Se valorará la inclusión de gráficos explicativos.
- Análisis léxico
 - o Descripción del archivo del lexer para definir y reconocer los componentes léxicos.
- Análisis sintáctico:
 - o Descripción del archivo del parser utilizado para definir la gramática libre de contexto.
- Funciones auxiliares
 - o Se deben indicar y describir las funciones auxiliares que se hayan codificado.
- Modo de obtención del intérprete
 - o Nombre y descripción de cada archivo utilizado
- Modo de ejecución del intérprete
 - o Interactiva
 - o A partir de un archivo
- Conclusiones:
 - o Reflexión sobre el trabajo realizado.
 - o Puntos fuertes y puntos débiles del intérprete desarrollado.
- Bibliografía o referencias web.
- Anexos



o Se podrían incluir aquellos anexos que se consideren oportunos para mejora la calidad de la documentación

Video Educativo / Promocional:

Resumen audiovisual de hasta 7 minutos, relatando y presentando con sus palabras el trabajo realizado por el equipo.

Debe resumir y destacar lo documentado en el Informe final.

Debe ser publicado de manera permanente en alguna plataforma de videos (youtube, vimeo, tiktok,etc)

8. Presentación y Criterios de evaluación

Documentación: 40 %

Se tendrá en cuenta lo indicado en el apartado no 7.

o El código elaborado deberá estar documentado.

o Se valorará la inclusión de gráficos o figuras.

o Se valorará la cantidad, originalidad y complejidad de los ejemplos propuestos.

o También se valorará:

la acentuación,

la corrección ortográfica

y la calidad y claridad de la redacción.

o Conocimiento que cada integrante del grupo aporte durante la exposición final del trabajo.

Funcionamiento del intérprete (software): 60 %

o El intérprete deberá:

funcionar correctamente tanto de forma interactiva como ejecutando las instrucciones desde los archivos de ejemplo.

Indicar como salida si el análisis fue exitoso (archivo correctamente codificado, sin errores) , en otro caso indicar los errores existentes (indicando tipo de error , número de línea y cadena que generó el error)

Traducir el documento, generando un archivo de texto HTML correcto.

en particular, deberá evaluar y traducir correctamente el ejemplo propuesto por el grupo y dos ejemplos más: uno al azar de los presentados por otros grupos y otro ejemplo elaborado por la cátedra.

o Se valorará

la completitud del lenguaje de código.

La calidad en el diseño del lenguaje y la gramática.

El control de errores.

La ampliación de elementos del lenguaje.

Observación:

Además, se valorará la asistencia a clase de prácticas y la resolución de dificultades encontradas durante la elaboración del trabajo.

9. Ejemplos

Básico

```
{ "equipos": [  
  {
```



```

"nombre_equipo": "Sintaxis y semantica de Lenguajes",
"identidad_equipo":
"https://www.frre.utn.edu.ar/public/themes/utnsd/images/UTN_FRRE-isi.png",
"link": "https://frre.cvg.utn.edu.ar/course/view.php?id=399",
"asignatura": "Sintaxis y semantica de Lenguajes",
"carrera": "Ingenieria en sistemas de Informacion",
"universidad_regional": "UTN Facultad Regional Resistencia",
"dirección": {
  "calle": "French 414",
  "ciudad": "Resistencia",
  "país": "argentina"
},
"alianza_equipo": "La organizacion del equipo se realiza segun la
planificacion anual, junto al cronograma de clases. Alli se determinan los horarios
de clases, temas a tratar cada clase y resultados esperados. La comunicacion entre
integrantes se realiza mediante correo electronico, grupos de whatsapp",
"integrantes": [
  {
    "nombre": "Rodrigo Vigil",
    "edad": 30,
    "cargo": "Marketing",
    "foto": "https://pbs.twimg.com/media/FCPNWd8XsAg5AkL.jpg",
    "email": "rodrigo@chirigui.com.mx",
    "habilidades": "receptor, bateador",
    "salario": 57000.45,
    "activo": true
  },
  {
    "nombre": "Juliana Torre",
    "edad": 22,
    "cargo": "Project Manager",
    "foto":
"https://i.pinimg.com/736x/96/89/97/9689977519645c642236582b6dc64c33.jpg",
    "email": "juliana@bears.com",
    "habilidades": "receptor, lanceador",
    "salario": 81300.45,
    "activo": true
  },

```



```
{
  "nombre": "Nicolas Tortosa",
  "edad": 25,
  "cargo": "DeveLoper",
  "foto":
    "https://www.baseballamerica.com/wp-content/uploads/2025/02/Brian_OConnor_Virginia_
    Getty.jpg",
  "email": "nico@bears.com",
  "habilidades": "receptor, lanceador",
  "salario": 111300.75,
  "activo": true
} ],
"proyectos": [
  {
    "nombre": "Interprete JSON y traductor HTML",
    "estado": "In progress",
    "resumen": "construir analizadores lexico y sintactico que
    permita analizar, validar y transformar un documento JSON que describe y almacena
    informacion de equipos y proyectos. Debera indicar si esta bien construido
    (adecuado al formato indicado, sin errores) de otra manera indicar Los errores;
    adicionalmente a medida que analiza el documento debera transformar el contenido en
    un documento HTML.",
    "tareas": [
      {
        "nombre": "Gramatica",
        "estado": "In progress",
        "resumen": "Presentar La gramatica del analizador
        sintactico. contemplando restricciones del enunciado.",
        "fecha_inicio": 2025-03-18,
        "fecha_fin": 2025-05-04
      },
      {
        "nombre": "LEXER",
        "estado": "To Do",
        "resumen": "Presentar Lexer reconociendo todos Los
        tokens.Actualizar documento",
        "fecha_inicio": 2025-04-10,
        "fecha_fin": 2025-06-08
      }
    ]
  }
]
```



```
    },  
    {  
      "fecha_inicio": 2025-04-18,  
      "nombre": "PARSER",  
      "estado": "To Do",  
      "resumen": "Presentacion final. LEXER + PARSER .  
funcionamiento completo.Actualizar documento, subir video resumen.",  
      "fecha_fin": 2025-07-06  
    }],  
    "fecha_inicio": 2025-03-18,  
    "fecha_fin": 2025-07-06,  
    "video": "https://www.youtube.com/watch?v=BFL2GHu1S_I",  
    "conclusion": "por escribirse",  
  } ]  
}  
],  
"version": "Release 0.9",  
"firma_digital": string  
}
```